Enterprise E-Commerce Management System

Project Type :- Internship Project

College :- Karmaveer Bhaurao Patil College of Engineering, Satara, Maharashtra, India.

Internship :- Symbiosis Powered by Capgemini

Team Information :-

Role	Member Name
Backend Developer	Apurva Mane
Backend Developer	Shivani Mane
Backend Developer	Sanika Kamble
Backend Developer	Vaishali Sargar
Backend Developer	Divya Ingale
Guide/Mentor	Mahajan Mungal

Abstract :

The Enterprise E-Commerce Management System Backend is a scalable and modular web application developed using Spring Boot. The system is designed to manage various e-commerce operations such as user management, company management, product management, order processing, payment handling, shipping management, customer feedback, and support services.

The application follows a layered architecture consisting of Controller, Service, Service Implementation, Repository, and Database layers. This architecture ensures proper separation of responsibilities, making the system easy to maintain, test, and scale.

The project utilizes Spring Data JPA and Hibernate ORM for database operations and MySQL as the backend database. All functionalities are exposed through RESTful APIs, which can be tested using Postman or Swagger UI.

Keywords:- Spring Boot, REST API, Hibernate, JPA, MySQL, E-Commerce, Layered Architecture.

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Enterprise E-Commerce Management System Backend is developed to provide a centralized platform for managing all major activities involved in an e-commerce business.

The system handles:

- User Registration and Login
- Company Management
- Product and Category Management
- Order Processing
- Payment Management
- Shipping and Tracking
- Customer Feedback
- Customer Support

The application follows enterprise-level software development practices and provides a scalable backend architecture suitable for real-world business applications.

1.2 Objectives

The main objectives of the project are:

- To develop a scalable e-commerce backend system.
- To implement CRUD operations for all entities.
- To design RESTful APIs for communication.
- To manage data efficiently using Spring Data JPA and Hibernate.
- To maintain clean code using layered architecture.
- To ensure future scalability and maintainability.

CHAPTER 2: TECHNOLOGY STACK

Programming Language :

Java 17 is used as the primary programming language for implementing business logic and application functionality.

- **Framework**

Spring Boot is used for rapid application development and auto-configuration.

- **Database**

MySQL is used as the relational database management system for storing application data.

- **ORM Framework**

Hibernate ORM and Spring Data JPA are used to perform database operations and manage entity relationships.

- **Build Tool**

Maven is used for dependency management and project build automation.

- **Testing Tools**

Postman and Swagger UI are used for API testing and documentation.

- **Version Control**

Git and GitHub are used for source code management and collaboration.

CHAPTER 3: SYSTEM ARCHITECTURE

Layered Architecture :

The project follows a layered architecture consisting of five layers:

1. Controller Layer :
The Controller Layer receives HTTP requests from clients and returns HTTP responses.

Responsibilities:

- Accept client requests
- Validate inputs
- Return response objects

2. Service Layer :
The Service Layer defines business operations and service contracts.

Responsibilities:

- Business process definition
- Method declarations
- Workflow management

3. Service Implementation Layer :
This layer contains the actual implementation of business logic.

Responsibilities:

- Process business rules
- Call repository methods
- Handle application logic

4. Repository Layer :
The Repository Layer communicates directly with the database.

Responsibilities:

- Insert data
- Update data
- Delete data
- Retrieve data

5. Database Layer :
MySQL stores all persistent data used by the application.

CHAPTER 4: APPLICATION WORKFLOW

The request-response workflow follows these steps:

1. The client sends an HTTP request.
2. The Controller receives the request.
3. The Controller forwards the request to the Service Layer.
4. The Service Layer processes business logic.
5. The Repository Layer interacts with the database.
6. The database returns the requested data.
7. The response is sent back to the client.

Workflow:

User

↓

Controller

↓

Service

↓

Repository

↓

Database

↑

Repository

↑

Service

↑

Controller

↑

User

This structured workflow improves maintainability and simplifies debugging.

CHAPTER 5: DATABASE DESIGN

The Enterprise E-Commerce Management System consists of 31 entity classes representing different modules of the system. These entities are mapped to database tables using JPA and Hibernate annotations.

The database is designed to manage users, companies, products, orders, payments, shipping, customer support, and geographical information efficiently.

The relationships between entities are implemented using One-to-One, One-to-Many, Many-to-One, and Many-to-Many mappings.

Entity Classification :

A. User and Access Management :
These entities manage system users and their roles.

1. User
2. Admin
3. Role
4. Address

Functions:

- User Registration
- User Login
- Role Management
- Address Management

B. Company Management :
These entities manage organizational structure and company information.

5. Owner
6. Company
7. Type
8. Manager
9. Dept
10. Employee

Functions:

- Company Creation
- Company Type Management
- Department Management
- Employee Management
- Manager Assignment

C. Product Management :
These entities manage product catalog information.

11. Category
12. SubCategory
13. Brand

14. Product
15. ProductReview

Functions:

- Category Management
- Product Management
- Brand Management
- Product Reviews

D. Order and Payment Management:
These entities handle order processing and payment operations.

16. Order
17. PaymentMode
18. Card
19. Upi
20. Cod
21. Invoice

Functions:

- Order Placement
- Payment Processing
- Invoice Generation
- Payment Mode Management

E. Shipping and Delivery Management :
These entities manage order shipment and tracking.

22. ShippingDetails
23. Tracking

Functions:

- Shipping Information
- Delivery Tracking
- Order Status Updates

F. Customer Support Management :
These entities handle customer communication and feedback.

24. Feedback
25. CustomerQuery
26. CompanyResponse

Functions:

- Customer Feedback
- Query Management
- Company Responses

G. Location Management :
These entities represent geographical hierarchy used throughout the system.

27. Country
28. State
29. District
30. Taluka
31. Town

Functions:

- Country Management
- State Management
- District Management
- Taluka Management
- Town Management

CHAPTER 6: REST API IMPLEMENTATION

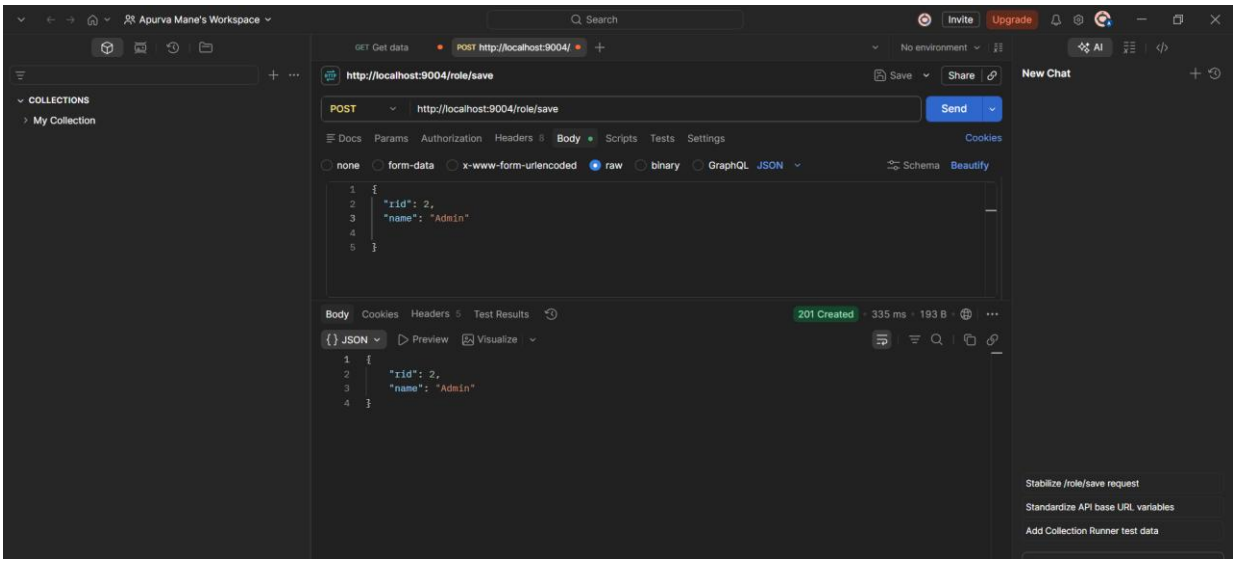
The application exposes REST APIs for all entities.

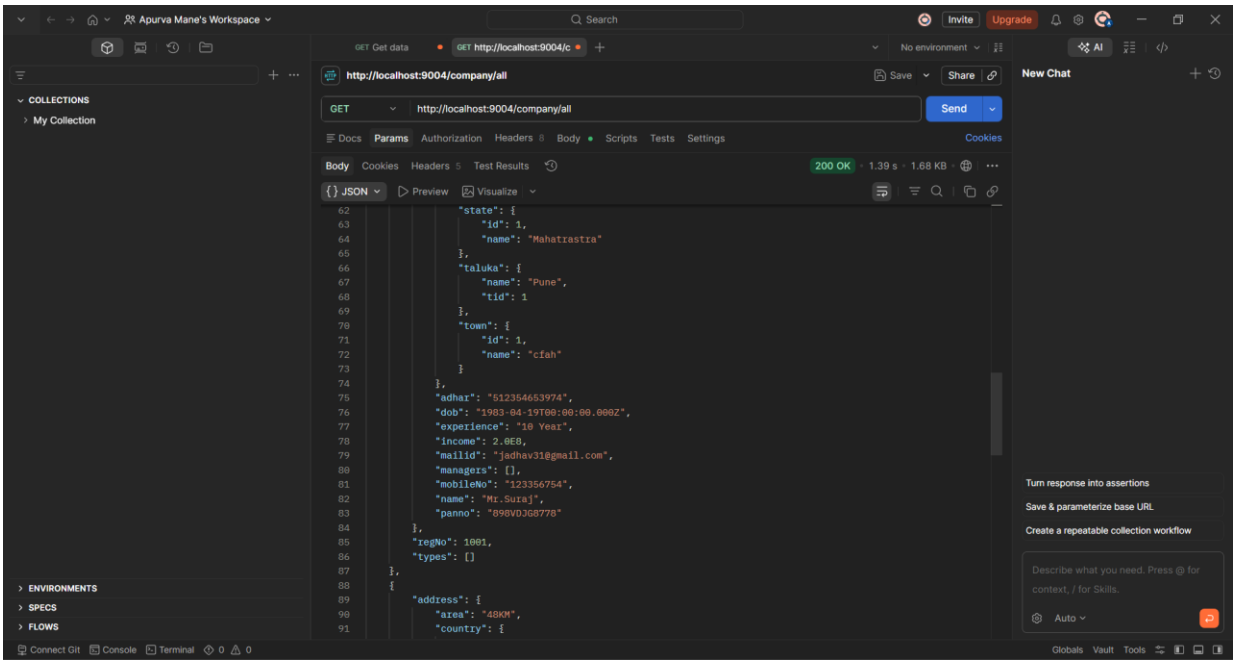
- **GET :** Used to retrieve data from the database.
- **POST :** Used to create new records.
- **PUT :** Used to update existing records.
- **DELETE :** Used to remove records.

These APIs are tested using Postman and Swagger UI.

CHAPTER 7: SNAPSHOTS

- **Postman :**





- **Database (Tables):**

```
| card                       |
| cash_on_delivery           |
| category                   |
| cod                        |
| company                    |
| company_response           |
| company_response_c_query   |
| country                    |
| customer_query             |
| dept                       |
| distric                    |
| employee                   |
| feedback                   |
| feedback_c_response        |
| feedback_cquery            |
| invoice                    |
| invoice_product            |
| invoice_shipping_details   |
| manager                    |
| order_product              |
| orders                     |
| owner                      |
| owner_managers             |
| payment_mode               |
| product                    |
| product_review             |
| role                       |
| shipping_details           |
| shipping_details_user      |
| state                      |
| sub_category               |
| taluka                     |
| town                       |
| tracking                   |
| type                       |
| upi                        |
| user                       |
+----------------------------+
40 rows in set (0.00 sec)

mysql>
```

CHAPTER 8: KEY FEATURES

The major features of the system include:

- Product Management
- Category Management
- Brand Management
- Company Management
- Order Processing
- Payment Handling
- Invoice Generation
- Shipping Management
- Order Tracking
- Customer Feedback
- Customer Support

Additional Features:

- Layered Architecture
- RESTful APIs
- Dependency Injection
- Hibernate ORM
- Spring Data JPA Integration
- Scalable Design
- Maintainable Code Structure

CHAPTER 9: FUTURE ENHANCEMENTS

The following features can be added in future versions:

- Spring Security with JWT Authentication
- Role-Based Access Control (RBAC)
- Email Notifications
- Payment Gateway Integration
- Docker Deployment
- AWS Cloud Deployment
- Microservices Architecture
- Advanced API Documentation

CONCLUSION

The Enterprise E-Commerce Management System Backend successfully demonstrates the implementation of enterprise-level backend development using Spring Boot. The project utilizes layered architecture, REST APIs, Hibernate ORM, Spring Data JPA, and MySQL to provide a scalable and maintainable solution for e-commerce operations.

The system effectively manages users, companies, products, orders, payments, shipping, and customer support processes while maintaining clean architecture and modular design principles.